System Design — Volume III

Deep Internals & Org-Scale Architecture | Principal Level

Apple Staff / Principal Engineer Interview Preparation

System Design Series — Printer-Friendly Edition

System Design — Volume III

Apple Inc Principal Engineer Interview Prep | Deep Internals & Org-Scale Architecture

■ = *Frequently asked at Apple* | ■ = *Advanced* | ■ = *Principal-level depth*

>

This volume assumes mastery of Vol I (Staff) and Vol II (Senior Staff). *Principal Engineer interviews shift from "can you design a system" to "can you reason about systems you've never seen, justify every trade-off quantitatively, foresee failure modes 3 years out, and influence org-wide decisions."*

What Makes Principal Engineer Different

```
Staff Engineer:         Design a URL shortener. Scale it to 1B requests/day.
Senior Staff Engineer:  Design Uber. Explain CAP, CRDT, Saga trade-offs.
Principal Engineer:     Design the platform that lets 50 teams build their own
                        Ubers safely, reliably, and cost-efficiently over 5 years.
                        What changes when you go from 1 region to 6?
                        How do you migrate 10B existing records with zero downtime?
                        What breaks at 10× scale you haven't hit yet?

Key differences in PE interviews:
  ✓ Depth over breadth — "explain exactly how Postgres MVCC prevents dirty reads"
  ✓ System evolution — "how does this design change in 2 years?"
  ✓ Org impact — "how do you roll this out to 40 teams without breaking them?"
  ✓ Failure mode exhaustion — "what's the blast radius if Kafka is down for 30 min?"
  ✓ Cost awareness — "this design costs $2M/month more — is it worth it?"
  ✓ Build vs buy — "Kafka vs Pulsar vs Kinesis for our specific workload — justify"
```

Table of Contents

Part B — Advanced Distributed Systems

Part C — Advanced Infrastructure

Part D — Principal-Level Design Problems

Part A — Storage Internals

Chapter 1: LSM Trees

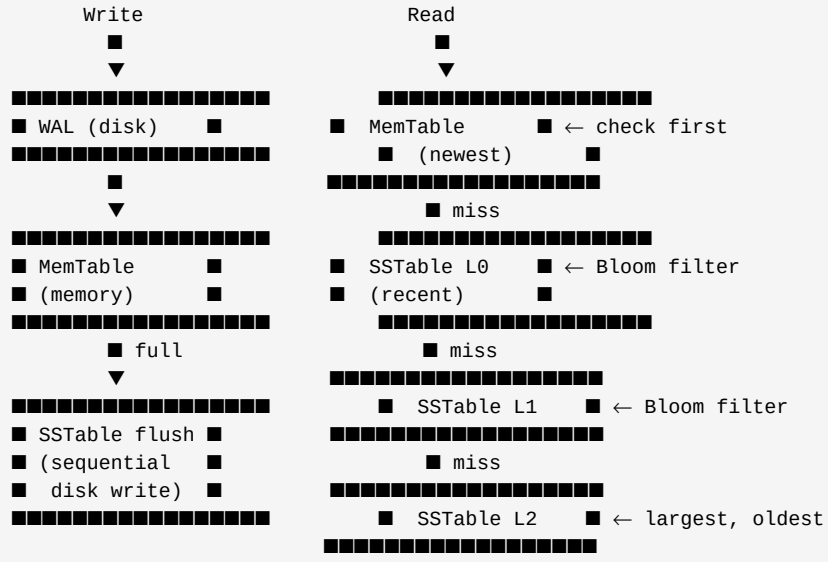
Q1 ■ ■ How does an LSM Tree work? Why do Cassandra and RocksDB use it?

Plain English First

A B-Tree updates data **in place** — like crossing out a word in a book and writing the new one. This is fast to read but slow to write (random I/O on disk).

An **LSM Tree (Log-Structured Merge Tree)** never overwrites. It always **appends** — like only adding new pages to a book. Appending is 10–100× faster on spinning disks and significantly faster on SSDs (sequential vs random writes). The trade-off: reads become more complex because the same key may exist across multiple files.

```
Write Path (extremely fast — always sequential):

1. Write to WAL (Write-Ahead Log)
   Append to wal.log: {key=user:123, value={name:"Alice"}, timestamp=t1}
   Crash safety: if server dies here, WAL replays on restart

2. Write to MemTable (in-memory sorted structure, usually a skip list)
   MemTable: {user:100→..., user:123→{name:"Alice"}, user:150→...}
   Instantly available for reads — zero disk I/O

3. When MemTable fills (e.g., 64MB):
   Flush to disk as an immutable SSTable (Sorted String Table)
   SSTable: sorted key-value file, written sequentially — very fast

4. Background Compaction (merges SSTables)
   Merge smaller SSTables into larger ones (like merge sort)
   Eliminates duplicate/stale keys (tombstones), reclaims space

Read Path (more complex):
1. Check MemTable (newest data — O(log n) in skip list)
2. Check each SSTable from newest to oldest (via Bloom filter first!)
   Bloom filter: "does this key MAYBE exist in this SSTable?"
   If Bloom filter says NO → skip this SSTable entirely (saves I/O)
   If Bloom filter says MAYBE → do binary search in SSTable index
3. Return first (newest) match found
```



Write Amplification vs Read Amplification vs Space Amplification

```
The RUM Conjecture: you can optimize for at most two of three:
  R — Read amplification  (how many I/Os to read one key)
  U — Update/Write amplification (how many I/Os to write one key)
  M — Memory/Space amplification (how much extra space used)

B-Tree:
  Read amp:   O(log N) — follow tree levels to leaf
  Write amp:  O(log N) — update in place (COW pages)
  Space amp:  ~1×      — near-minimal (no duplicates)
  Best for: read-heavy workloads

LSM Tree:
  Read amp:   O(levels) — check MemTable + each SSTable level
              Mitigated by Bloom filters (skip most SSTables)
  Write amp:  O(levels × size_ratio) — data rewritten on each compaction level
              Typically 10–30× (written to disk many times before final resting place)
  Space amp:  ~2×       — old versions kept until compaction removes them
  Best for: write-heavy workloads (time-series, logs, IoT)

Compaction strategies:
  Size-Tiered (Cassandra default):
    Merge SSTables of similar size
    Low write amplification, high space amplification (multiple versions coexist)
    Good for: write-heavy, insert-only (time-series)

  Leveled (RocksDB default, LevelDB):
    Each level has one SSTable per key range, L(n) is 10× size of L(n-1)
    Lower space amplification, higher write amplification
    Good for: read-heavy workloads that need low space usage

  FIFO (RocksDB option):
    Delete oldest SSTables when storage full
    No compaction overhead, data naturally expires
    Good for: cache-like time-series where old data discarded
```

Chapter 2: B-Tree Internals & Crash Recovery

Q2 ■ How does a B-Tree database (PostgreSQL) survive a crash? What is WAL?

```
B-Tree structure:
  Self-balancing tree where each node = one disk page (typically 8KB)
  Root → Internal nodes (keys + child pointers) → Leaf nodes (keys + row data)
  Depth: ~4 levels for 1B rows (log_B(N) where B = branching factor ~100)
  Read: 4 page reads for any key lookup (predictably fast)
  Write: find leaf, update in-place, propagate splits if needed

The crash problem:
  Writing a single row update may touch 3–4 disk pages:
  1. Update the data page (leaf node)
  2. Update the parent index page
  3. Update the free space map
  If server crashes between step 1 and 3: data is inconsistent (torn write)

Write-Ahead Log (WAL) — the solution:
  Rule: ALWAYS write to WAL BEFORE modifying the data page

  WAL entry: { LSN: 1234, transaction_id: 456, operation: UPDATE, page: 789,
               before_image: <old bytes>, after_image: <new bytes> }

  Write path:
  1. Write WAL entry to WAL buffer (in memory)
  2. Flush WAL buffer to WAL file on disk (fsync — durability guarantee)
  3. THEN modify the actual data page (may stay in memory for a while)

  Crash recovery:
  On restart PostgreSQL reads WAL from last checkpoint:
    REDO: re-apply all committed transactions found in WAL
    UNDO: roll back any transactions that were in-progress (no COMMIT record found)
  Result: DB is always in a consistent state after restart

Checkpoint:
  Periodically (every 5 min by default), PostgreSQL flushes all dirty pages to disk
  Records checkpoint LSN in WAL
  On restart: only need to replay WAL from last checkpoint (not entire history)
  Trade-off: frequent checkpoints = fast recovery but I/O spike during checkpoint
```

Chapter 3: MVCC

Q3 ■ ■ How does MVCC work in PostgreSQL? How does it prevent dirty reads without locks?

Plain English First

In a traditional locking system: when you read a row, you lock it — nobody can write while you read. This kills concurrency.

MVCC (Multi-Version Concurrency Control) keeps **multiple versions** of each row. Readers see an older consistent snapshot; writers create new versions. Readers never block writers; writers never block readers.

```
The core idea:
  Every row has xmin (transaction that created it) and xmax (transaction that deleted it)

  Row version 1: {user_id:1, name:"Alice", xmin: tx100, xmax: NULL}
  Transaction tx200 updates name to "Bob":
  Row version 2: {user_id:1, name:"Bob",   xmin: tx200, xmax: NULL}
  Row version 1 becomes: {user_id:1, name:"Alice", xmin:tx100, xmax:tx200}

Transaction visibility rules:
  A row version is VISIBLE to transaction T if:
  - xmin is a committed transaction AND xmin < T's snapshot
  - xmax is NULL (not deleted) OR xmax > T's snapshot OR xmax aborted

  Transaction tx300 (started after tx200 committed):
    Sees: row version 2 (name="Bob") — correct!

  Transaction tx150 (started before tx200):
    Sees: row version 1 (name="Alice") — its snapshot is from before tx200
    No locks held by either transaction — they proceed concurrently!

Snapshot Isolation:
  Each transaction gets a snapshot at its start time
  All reads see the DB as it was at snapshot time
  Writes create new versions — don't modify old ones

  Anomaly: Write Skew (snapshot isolation doesn't prevent it)
  Doctor scheduling example:
    tx1: reads "2 doctors on call", decides to go off call
    tx2: reads "2 doctors on call", decides to go off call
    Both commit: 0 doctors on call — WRONG
  Fix: Serializable Snapshot Isolation (SSI) — PostgreSQL's "SERIALIZABLE" level

VACUUM — the cleanup job:
  Old row versions accumulate (dead tuples that no transaction can see)
  VACUUM scans tables, marks dead tuples as reusable space
  Without VACUUM: table bloats, wraps around (transaction ID wraparound — catastrophic)
  autovacuum: background daemon that runs VACUUM automatically

Transaction ID Wraparound (catastrophic failure mode):
  PostgreSQL uses 32-bit transaction IDs (4 billion values)
  If IDs wrap around without freezing old rows: ALL rows appear to be in the future → invisible
  pg_freeze: marks old rows as "frozen" (always visible, exempt from wraparound)
  Monitor: SELECT max_age FROM pg_stat_user_tables ORDER BY max_age DESC
  Alert if max_age approaches 2 billion (halfway to wraparound)
```

Chapter 4: Probabilistic Data Structures

Q4 ■ ■ What are Bloom Filters, HyperLogLog, and Count-Min Sketch? When do you use them?

Plain English First

Sometimes exact answers cost too much memory. These data structures trade **a small, bounded error** for **dramatically less memory**. A principal engineer reaches for these when exact computation is impossible at scale.

Bloom Filter — "Does this key exist?"

```
Answers: DEFINITELY NOT (100% accurate) or PROBABLY YES (small false positive rate)
Never has false negatives — if it says "no", the key truly doesn't exist

How it works:
  Bit array of M bits, all initialized to 0
  K hash functions: h1, h2, ..., hk

  INSERT "apple":
    h1("apple") = 3  → set bit[3] = 1
    h2("apple") = 7  → set bit[7] = 1
    h3("apple") = 14 → set bit[14] = 1

  QUERY "banana":
    h1("banana") = 5  → bit[5] = 0 → DEFINITELY NOT in set (stop here)

  QUERY "apple":
    h1("apple") = 3  → bit[3] = 1 ✓
    h2("apple") = 7  → bit[7] = 1 ✓
    h3("apple") = 14 → bit[14] = 1 ✓
    → PROBABLY YES (could be false positive from other keys setting same bits)

Memory: ~10 bits per element for 1% false positive rate
  1 billion URLs → ~1.2 GB (vs 25 GB for a HashSet of 8-byte longs)
  1% false positive rate → 1 in 100 queries incorrectly returns "probably yes"

Real uses:
  Cassandra: before reading SSTable from disk, check Bloom filter
    "Does key user:123 exist in this SSTable?"
    If NO → skip disk read entirely (major read amplification reduction)

  Google BigTable: Bloom filter per SSTable
  CDNs: "has this URL been cached?" — avoid DB lookup for cold URLs
  Chrome Safe Browsing: "is this URL malicious?" — local Bloom filter, only call server on "maybe"
  Weak password detection: "is this password in the list of 100M known weak passwords?"
```

HyperLogLog — "How many unique visitors?"

```
Problem: count distinct elements in a stream (cardinality estimation)
Exact solution: store every element in a HashSet → O(N) memory
HyperLogLog: estimate with < 1% error using O(log log N) memory — 12KB for any cardinality!

How it works (simplified):
  Hash each element → random-looking binary string
  Track the maximum number of leading zeros seen
  Intuition: if max leading zeros = k, you've probably seen ~2^k unique elements

  Element "user123" → hash → 0001001010...
  Leading zeros: 3 → 2^3 = 8 estimate

  More elements → more likely to see rare leading-zero sequences → higher estimate

Accuracy: ±2% error with 1.5KB memory, ±1% with 12KB
  Exact: 1B unique users × 8 bytes (pointer) = 8 GB
  HLL:   1B unique users → 12 KB — that's 666,000× less memory!

Real uses:
  Redis: PFADD, PFCOUNT — built-in HyperLogLog commands
  Analytics: "how many unique users visited this page today?"
  Databases: query optimizer uses cardinality estimates for join planning
  Kafka: estimate unique producers/consumers

  daily_active_users_estimate = PFCOUNT("users:2026-05-07")
  PFADD("users:2026-05-07", user_id)   // O(1) per event
  PFCOUNT("users:2026-05-07")          // Always O(1), always 12KB regardless of DAU count
```

Count-Min Sketch — "How many times has X appeared?"

```
Problem: count frequency of each element in a stream
Exact: HashMap<String, Long> → O(N) memory
CMS: approximate frequency with bounded error, O(1) memory

How it works:
  Grid of W columns × D rows, all initialized to 0
  D independent hash functions: h1, h2, ..., hd

  INCREMENT "buy:productA":
    h1("buy:productA") = col 3, row 1 → matrix[1][3]++
    h2("buy:productA") = col 7, row 2 → matrix[2][7]++
    h3("buy:productA") = col 2, row 3 → matrix[3][2]++

  QUERY "buy:productA":
    Return MIN(matrix[1][3], matrix[2][7], matrix[3][2])
    Multiple cells — take minimum (overcounting from hash collisions reduced by min)

Error bound: With W=2000 columns, D=7 rows:
  Estimate is always ≥ true count (never undercounts)
  Overcount ≤ total_events / W = total / 2000 per query
  Error probability ≤ (1/2)^D = (1/2)^7 < 1%
  Memory: 7 × 2000 × 8 bytes = ~112 KB regardless of stream size

Real uses:
  Trending hashtags: estimate tweet count per hashtag (can't store counts for all hashtags)
  Network traffic: count packet frequency per (src_ip, dst_ip) pair — DDoS detection
  Recommendation systems: estimate item interaction frequency
  Rate limiting at edge: approximate per-user request count without exact storage
```

Chapter 5: Columnar Storage & Query Optimization

Q5 ■ ■ Why is columnar storage (Parquet, ORC) dramatically faster for analytics?

```
Row-oriented (PostgreSQL, MySQL):
  Stores all columns of a row together on disk
  Row 1: [id=1, name="Alice", age=30, salary=120000, dept="Eng"]
  Row 2: [id=2, name="Bob",   age=25, salary=95000,  dept="Mkt"]

  Query: SELECT AVG(salary) FROM employees WHERE dept = 'Eng'
  Must read EVERY column of EVERY row — even "name" and "age" which aren't needed
  For 100M rows × 100 columns = 100B values read to compute one AVG

Column-oriented (Parquet, ORC, ClickHouse, Redshift):
  Stores all values of one column together on disk
  salary: [120000, 95000, 87000, 135000, ...]   ← all salaries, contiguous
  dept:   ["Eng", "Mkt", "Eng", "Eng", ...]     ← all depts, contiguous

  Query: SELECT AVG(salary) FROM employees WHERE dept = 'Eng'
  1. Read dept column, find row positions where dept = 'Eng'
  2. Read salary column for ONLY those positions
  Total I/O: 2 columns × relevant rows — 98 columns never touched!

Performance advantages:
  1. Column pruning: read only needed columns (vs all columns)
  2. Compression: similar values together compress extremely well
     salary column: [120000, 95000, 87000, ...] → delta encoding (store differences)
     dept column:   ["Eng","Mkt","Eng","Eng"] → dictionary encoding ("Eng"=0, "Mkt"=1, ...)
     Typical: 5-10× better compression than row-oriented
  3. Vectorized execution: process 1024 values at once using SIMD CPU instructions
     CPU executes AVG on a vector of salary integers in one instruction
     100-1000× faster than row-by-row processing

Parquet file format:
  File → Row Groups (128MB) → Column Chunks → Pages (1MB)
  Each Row Group has column statistics: min, max, null_count
  Query planner: if salary_min > 200000 in a Row Group and we filter salary < 100000
    → skip entire Row Group (predicate pushdown — never read it from disk)

  RowGroup1: salary min=50000, max=150000 → might have rows matching salary < 100000 → read
  RowGroup2: salary min=200000, max=500000 → can't have salary < 100000 → SKIP entirely

ClickHouse (real-time columnar OLAP):
  10B rows/sec ingestion
  Sub-second queries on billions of rows
  Sparse indexes: one index entry per 8192 rows
  MergeTree engine: similar to LSM Tree but for columns
  Used for: Apple's analytics, Cloudflare's logging, Uber's trip analytics
```

Part B — Advanced Distributed Systems

Chapter 6: Google Spanner & TrueTime

Q6 ■ ■ How does Google Spanner achieve globally distributed ACID transactions?

Plain English First

Spanner is the database that shouldn't exist — it's a globally distributed SQL database with ACID transactions and strong consistency (linearizability), spanning dozens of data centers across the planet.

Traditional wisdom: you can't have both global distribution AND strong consistency (CAP theorem says pick availability OR consistency during a partition).

Spanner's insight: **if you can synchronize clocks precisely enough**, you can prove that one transaction committed before another — without waiting for cross-datacenter round-trips.

```
The problem: ordering events across data centers

  San Jose DC:   tx1 commits at local time 10:00:00.000 (Pacific)
  Dublin DC:     tx2 commits at local time 18:00:00.005 (IST, but same real moment)

  How do you know which happened first? You can't trust local clocks
  (clock drift between data centers can be 100ms–10ms)
  If you trust local clocks: "tx1 before tx2" could be wrong → stale reads

TrueTime — the solution:
  Google deploys GPS receivers and atomic clocks in every data center
  TrueTime API returns: [earliest, latest] interval within which "true time" lies
  Uncertainty (ε): typically 1–7ms (guaranteed upper bound on clock uncertainty)

  TT.now() returns: { earliest: t-ε, latest: t+ε }
  You KNOW the true time is within this interval

Commit Wait — how Spanner uses TrueTime:
  When transaction T wants to commit:
  1. Compute a commit timestamp: s = TT.now().latest
  2. WAIT until TT.now().earliest > s  (wait for uncertainty to pass)
  3. NOW you are CERTAIN that no future transaction can have a timestamp < s
     (the "true time" is definitely past s)
  4. Commit with timestamp s — guaranteed to be in the right order globally

  Cost: wait ~2ε = 2-14ms extra per commit (worth it for global consistency)

Paxos Group per Shard:
  Data is range-sharded (like BigTable)
  Each shard: one Paxos group (across 5 replicas in different regions)
  Writes: go to Paxos leader (strong consistency within shard)
  Reads (stale): can read from any replica (low latency, 1-10ms)
  Reads (strong): leader reads at a safe timestamp (7-14ms)

Read-Write Transactions (across shards):
  Uses Two-Phase Commit (2PC) across Paxos groups
  2PC coordinator: one of the shard leaders
  Locks held during 2PC are short (milliseconds) because TrueTime prevents stale reads
  Without TrueTime: 2PC must wait for cross-region RTT confirmation (50-150ms)
  With TrueTime: wait only 2ε (2-14ms) — 10× faster than naive cross-region 2PC

CockroachDB / YugabyteDB (open-source Spanner-like):
  No GPS/atomic clocks → use Hybrid Logical Clocks (HLC)
  HLC = max(physical_time, logical_time) + counter
  Uncertainty: 500ms (much larger than TrueTime's 7ms)
  On potential conflict: wait out the uncertainty window (or resolve via gossip)
  Trade-off: less precise clock → more conservative waiting → slower global transactions
```

Chapter 7: Cell-Based Architecture

Q7 ■ ■ What is Cell-Based Architecture? How does it limit blast radius?

Plain English First

When your entire user base shares one cluster, a bug, deploy, or traffic spike can take down the **entire service**. Cell-based architecture divides your infrastructure into isolated "cells", each serving a fraction of users. A failure in one cell affects only that cell's users, not everyone.

Think of it like a submarine's watertight compartments — a hull breach floods one compartment, not the entire ship.

```
Without cells (monolithic cluster):
  All 100M users → Single Kafka cluster → Single DB cluster
  Bad deploy → 100M users down
  Traffic spike in US → EU users also degrade

With cell-based architecture:
  Cell A (users 0–25M)   → Kafka_A → DB_A → Services_A
  Cell B (users 25M–50M) → Kafka_B → DB_B → Services_B
  Cell C (users 50M–75M) → Kafka_C → DB_C → Services_C
  Cell D (users 75M–100M)→ Kafka_D → DB_D → Services_D

  Bad deploy to Cell A → 25M users affected (not 100M)
  Fix Cell A, resume → roll deploy to Cell B, C, D
  Traffic spike in US (Cell A) → Cell B, C, D unaffected
```

Cell Routing

```
Cell assignment:
  user_id % num_cells → cell_id
  OR: geographically (US users → US cells, EU → EU cells)
  OR: tenant-based (each enterprise customer gets a dedicated cell)

Cell Router (DNS / Load Balancer):
  User request arrives → Router checks user_id → routes to correct cell
  Router must be highly available (it's in the critical path)
  Cell map stored in a distributed config store (etcd / ZooKeeper)

  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■                    Cell Router                         ■
  ■  user_id → hash → cell_id → cell_endpoint              ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
           ■              ■               ■
    ■■■■■■■▼■■■■■■■ ■■■■■■▼■■■■■■■■ ■■■■■▼■■■■■■■■■■
    ■   Cell A    ■ ■   Cell B    ■ ■   Cell C    ■
    ■ (users 0-25M■ ■ (users 25M- ■ ■ (users 50M- ■
    ■  Kafka_A    ■ ■  Kafka_B    ■ ■  Kafka_C    ■
    ■  DB_A       ■ ■  DB_B       ■ ■  DB_C       ■
    ■  Services_A)■ ■  Services_B)■ ■  Services_C)■
    ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■
```

Cross-Cell Operations

```
Problem: what if user in Cell A sends a message to user in Cell B?

Option 1: Route through shared message bus (Kafka topic per pair of cells)
  Cell A publishes to "cell-a-to-cell-b" topic
  Cell B consumes from same topic
  Async, eventual delivery

Option 2: Direct API call (Cell A service calls Cell B service)
  Synchronous, immediate, but creates cross-cell dependency
  If Cell B is down: does Cell A degrade too? (partial blast radius failure)

Option 3: Replicated shared data (read replicas across cells)
  User profile replicated to all cells → no cross-cell reads for profile lookups
  Only write to home cell → replicate out
  Trade-off: storage cost × num_cells

Apple's iMessage is cell-based:
  Message delivery to another user's cell via async message bus
  Each cell is independently deployable and scalable
  Incident in one region doesn't affect other regions
```

Cell Operations

```
Rolling deploys across cells:
  Deploy to Cell A (25M users) → monitor for 30 min
  No alerts? Deploy to Cell B → monitor
  Alert fires? Roll back Cell B only, investigate with Cell A as canary

Cell health scoring:
  Each cell reports: error rate, P99 latency, queue depth
  Orchestrator: if cell_health < threshold → drain traffic from that cell
                → route its users to neighboring cells temporarily

Load shedding within a cell:
  Cell A traffic spikes beyond capacity
  Shed: non-critical background jobs stop first
  Shed: analytics events dropped
  Shed: retry-eligible requests rejected with 503 (client retries later)
  Preserve: financial transactions, auth, critical notifications
```

Chapter 8: Chaos Engineering & Resilience Testing

Q8 ■ ■ What is Chaos Engineering? How do you run it safely at Apple's scale?

```
Chaos Engineering: deliberately inject failures into a running production system
to discover weaknesses before they cause real outages.

"Break things on purpose, in a controlled way, before they break on their own."
— Netflix, who coined the term with Chaos Monkey

Why production (not staging)?
  Staging doesn't have real traffic patterns, real data volumes, or real failure modes
  Your system behaves differently under real production load
  Many failures only manifest under specific traffic conditions you can't replicate in staging

Hypothesis-driven experiments:
  NOT: "randomly kill things and see what happens"
  YES: "We HYPOTHESIZE that our system maintains < 1% error rate
        when one Kafka broker fails, because we have replication factor 3 and
        consumer group rebalancing. Let's VERIFY this is actually true."

Chaos experiment structure:
  1. Define steady state: system is healthy when error_rate < 0.1% AND P99 < 200ms
  2. Hypothesis: steady state maintained when [failure X] occurs
  3. Inject failure: kill one Kafka broker in Cell A
  4. Observe: does error_rate stay < 0.1%? Does P99 stay < 200ms?
  5. Stop experiment if metrics exceed thresholds (automated rollback)
  6. Document findings: hypothesis confirmed / refuted. What to fix.

Types of chaos experiments (escalating blast radius):
  Level 1 — Non-production
    Kill a staging service, verify circuit breakers trip
    Inject high latency on a staging DB, verify timeouts fire

  Level 2 — Production dark (canary)
    Kill one instance in one AZ, verify LB routes around it
    Inject 100ms latency into 1% of Redis calls, verify degraded-mode works

  Level 3 — Production with kill switch
    Kill entire AZ in one region, verify failover to another AZ
    Stop Kafka consumer group, verify message lag alert fires and on-call paged

  Level 4 — Regional (requires C-suite awareness)
    Simulate full US-East datacenter failure, verify failover to US-West
    Requires communication plan, customer comms template ready

Blast radius control:
  Feature flags: chaos engine only injects when flag is on
  Traffic shaping: only affect 1% of users first
  Automatic rollback: if error_rate > threshold, chaos engine stops automatically
  Time-boxing: experiments run for max 10 minutes automatically
  Business hour gating: only run during work hours when engineers are available
```

Game Days

```
Structured team exercises — not automated, human-driven scenario practice

Format:
  Duration: 4 hours
  Participants: on-call engineers, SREs, engineering leads
  Scenario: "Kafka cluster in US-East has failed. It's 2am. You get paged."
  Execution: chaos team silently injects failure at a random time in the window
  Team: works the incident using real runbooks, real tooling, real communication
  Retrospective: what worked? what was missing? update runbooks

Common game day scenarios at Apple scale:
  - Primary DB failover to replica (does app handle the reconnect?)
  - Redis cache wipe (does thundering herd protection work? does DB survive?)
  - CDN origin pulled offline (do users see CDN-cached content or errors?)
  - Certificate expiry (does your monitoring catch it 30 days before? What happens on expiry?)
  - Dependency service returns 5xx for 5 min (circuit breaker opens? fallback activates?)
  - Config store (etcd) is down (do services use last-known-good config? do they crash?)
  - Clock skew injected on 3 nodes (does distributed consensus break?)
```

Chapter 9: Schema Evolution Without Downtime

Q9 ■ ■ How do you change a database schema in production with zero downtime?

Plain English First

`ALTER TABLE users ADD COLUMN phone VARCHAR(20) NOT NULL` on a table with 500M rows — in many databases this **locks the entire table** for 30+ minutes. During that time: your app is down. At Apple scale, this is never acceptable.

The **Expand-Contract Pattern** (also called parallel change) lets you evolve schemas incrementally, with zero downtime and easy rollback at each step.

```
Scenario: rename column "email" to "email_address" in users table (500M rows)

WRONG approach:
  ALTER TABLE users RENAME COLUMN email TO email_address;
  → Table-level lock for minutes → downtime
  → Old app code reading "email" breaks immediately

RIGHT approach (5 steps over multiple deployments):

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Step 1: EXPAND — add the new column (no downtime)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ALTER TABLE users ADD COLUMN email_address VARCHAR(255);
  -- On PostgreSQL: adding nullable column is instant (no lock, no table rewrite)
  -- email_address is NULL for all existing rows

  Deploy app code v2:
    READS: read from "email" (old column) as before
    WRITES: write to BOTH "email" AND "email_address" (dual-write)

  Result: new writes populate both columns, old rows still NULL in new column

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Step 2: BACKFILL — populate new column for existing rows
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Background job (runs in batches to avoid lock contention):

  UPDATE users SET email_address = email
  WHERE email_address IS NULL
  AND id BETWEEN 0 AND 1000000;      -- batch 1 (sleep 10ms between batches)

  UPDATE users SET email_address = email
  WHERE email_address IS NULL
  AND id BETWEEN 1000000 AND 2000000; -- batch 2

  ... (500 batches for 500M rows, takes hours — zero downtime)

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Step 3: VERIFY — confirm backfill is complete
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  SELECT COUNT(*) FROM users WHERE email_address IS NULL;
  -- Must be 0 before proceeding

  Add NOT NULL constraint (online in PostgreSQL 12+ with NOT VALID trick):
  ALTER TABLE users ADD CONSTRAINT email_address_not_null
    CHECK (email_address IS NOT NULL) NOT VALID;
  -- NOT VALID: adds constraint without scanning table (instant)

  ALTER TABLE users VALIDATE CONSTRAINT email_address_not_null;
  -- Validates with ShareUpdateExclusiveLock (allows concurrent reads/writes!)
```

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Step 4: CONTRACT (part 1) — switch reads to new column
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Deploy app code v3:
    READS: read from "email_address" (new column)
    WRITES: still write to both (rollback safety — can revert to v2 if needed)

  Monitor: errors? performance issues? rollback to v2 if needed

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Step 5: CONTRACT (part 2) — drop old column
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  Deploy app code v4:
    WRITES: write to "email_address" only (stop writing to "email")

  Wait until no traffic is using old column (monitoring confirms)
  ALTER TABLE users DROP COLUMN email;  -- Safe now, instant
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Total time: days (backfill runs in background)
Downtime: 0 seconds
Rollback: available at every step (each step is independently reversible)
```

Schema Registry for Event-Driven Systems

```
Problem: Kafka consumers expect specific message formats
  If producer changes message schema, old consumers break

Avro + Schema Registry (Confluent):
  Each schema version registered in central Schema Registry
  Message contains schema_id (not full schema) → small overhead
  Consumer reads schema_id → fetches schema from Registry → deserializes

  Schema evolution rules (Avro):
  BACKWARD compatible: new schema can read old data
    → Add fields with defaults: OK (old messages missing field → use default)
    → Remove optional fields: OK (consumers ignore unknown fields)
    → Remove required fields: NOT OK (old messages lack the field)
    → Rename fields: NOT OK (old messages use old name)

  FORWARD compatible: old schema can read new data
    → Add fields without defaults: OK for forward
    → Remove fields: OK for forward

  FULL compatible: both backward AND forward compatible
    → Most restrictive, safest for long-running consumer groups

  Recommended: BACKWARD compatibility as minimum
  Register new schema → registry validates compatibility → deploy consumers → deploy producers
```

Part C — Advanced Infrastructure

Chapter 10: Service Mesh, mTLS & Zero-Trust

Q10 ■ ■ What is a service mesh? How does mTLS work between microservices?

Plain English First

In a naive microservice setup: Service A talks to Service B over plain HTTP on an internal network. Trust model: "if you're inside the VPC, you're trusted."

This is wrong. If an attacker compromises Service C (even a low-value service), they can impersonate any other service on the network — no authentication between services.

Zero-Trust: never trust, always verify. Every service-to-service call must be authenticated and encrypted, even on internal networks.

```
mTLS (Mutual TLS):
  Normal TLS: client verifies server's certificate (HTTPS)
  mTLS: BOTH sides verify each other's certificate

  Service A → Service B:
    B presents its certificate: "I am payment-service, cert signed by our CA"
    A verifies: cert is valid, signed by our trusted CA, name matches "payment-service"
    A presents its certificate: "I am order-service, cert signed by our CA"
    B verifies: cert is valid, authorized to call payment-service
    Connection established — encrypted + both sides authenticated

  Benefit: even if attacker is inside the VPC, they can't impersonate payment-service
  without its private key (which never leaves the payment-service pod)
```

Service Mesh — mTLS at Scale

```
Problem: implementing mTLS in 50 services means 50 teams each handle:
  Certificate generation, rotation, storage
  mTLS handshake code
  Certificate revocation
  Policy enforcement (who can call whom?)

Service Mesh (Istio + Envoy):
  Sidecar proxy pattern:
    Every pod gets an Envoy sidecar container injected automatically
    All traffic in/out of the pod goes through the Envoy sidecar
    Application code sees plain HTTP — sidecar handles mTLS transparently

  ┌──────────────────────────────────────┐  ┌──────────────────────────────────────┐
  │ Pod: Order Service                   │  │ Pod: Payment Service                 │
  │  ┌────────────────────┐              │  │  ┌────────────────────┐              │
  │  │ App (plain    ┌──┐ localhost ┌──┐ │  │  │ App (plain    ┌──┐ localhost ┌──┐ │
  │  │ HTTP :8080)   │  │           │  │ │  │  │ HTTP :8080)   │  │           │  │ │
  │  └────────────────────┘           │  │ │  │  └────────────────────┘           │  │ │
  │  ┌────────────────────┐           │  │ │  │  ┌────────────────────┐           │  │ │
  │  │ Envoy Sidecar ┌──┐ mTLS ┌──┐ Envoy Sidecar │  │           │  │ │
  │  │ (handles TLS, │  │      │  │      │  │  │           │  │ │
  │  │  auth, retry) │  │      │  │      │  │  │           │  │ │
  │  └────────────────────┘           │  │ │  │  └────────────────────┘           │  │ │
  └──────────────────────────────────────┘  └──────────────────────────────────────┘

Istio Control Plane:
  Distributes mTLS certificates to all Envoy sidecars (via SPIFFE/X.509)
  Rotates certificates automatically every 24 hours
  Enforces AuthorizationPolicy: "order-service CAN call payment-service"
                                "analytics-service CANNOT call payment-service"
  Telemetry: Envoy reports latency, errors, traffic to Prometheus/Jaeger

Zero-Trust Authorization Policy (Istio):
  apiVersion: security.istio.io/v1beta1
  kind: AuthorizationPolicy
  metadata: { name: payment-service-policy }
  spec:
    selector: { matchLabels: { app: payment-service } }
    action: ALLOW
    rules:
      - from:
          - source: { principals: ["cluster.local/ns/default/sa/order-service"] }
        to:
          - operation: { methods: ["POST"], paths: ["/charge", "/refund"] }

  Default: DENY ALL unless explicitly allowed
  Granularity: service identity + HTTP method + URL path
```

Chapter 11: HTTP/3, QUIC & Modern Networking

Q11 ■ How does HTTP/3 over QUIC improve performance? When does it matter at Apple's scale?

```
HTTP/1.1 → HTTP/2 → HTTP/3: evolution of the web protocol

HTTP/1.1 problems:
  One request per TCP connection → browsers open 6-8 connections per host
  Head-of-line blocking: request 2 waits for request 1 to complete

HTTP/2 improvements:
  Multiplexing: many requests/responses on one TCP connection (streams)
  Header compression (HPACK)
  Server push (server sends resources before client asks)
  Remaining problem: TCP-level head-of-line blocking
    If one TCP packet is lost → ALL streams wait for retransmit
    TCP doesn't know about application-level streams

HTTP/3 / QUIC (Google → IETF standard):
  Runs over UDP (not TCP) — QUIC implements its own reliable transport
  Stream-level reliability: packet loss only affects the one stream it belongs to
  Other streams continue unaffected (true multiplexing without HOL blocking)
  0-RTT handshake: returning clients can send data in the first packet
    (TLS 1.3 saves the session, first QUIC packet includes application data)
  Connection migration: if client's IP changes (switching from WiFi to cellular)
    QUIC connection persists (identified by Connection ID, not IP+port)
    TCP connection would break, requiring reconnect + TLS handshake

Performance impact:
  High packet loss (mobile, developing regions): HTTP/3 dramatically better
  Low packet loss (fast broadband): minimal difference
  0-RTT: ~100ms saved on repeat connections (critical for Apple Pay flow)

Apple's use:
  iOS devices switch between WiFi and cellular frequently
  QUIC connection migration: seamless handoff, no reconnect
  Apple CDN uses HTTP/3 for iOS software updates (large files, need reliability)

QUIC connection establishment:
  New connection:  1-RTT (client hello + server hello + data in next round-trip)
  Resumed connection: 0-RTT (client includes session ticket + data in first packet)
  vs TLS 1.2 over TCP: 3-RTT before first byte of data
  Savings: 200-400ms on mobile (2 × 100-200ms RTT saved)
```

Chapter 12: ML Infrastructure

Q12 ■ ■ What is a Feature Store? How does ML inference work at low latency?

Plain English First

Machine learning models need "features" — derived signals computed from raw data. The same feature (e.g., "user's average order value in last 30 days") is needed by 10 different models. Without a Feature Store, every team recomputes it differently — inconsistency between training and serving kills model accuracy.

```
The Training-Serving Skew problem (most common ML production failure):
  During training:  compute "avg_order_value_30d" using batch Spark job on historical data
  During serving:   a different team approximates it with a quick DB query
  Result: model trained on one distribution, serving on a different one → bad predictions

Feature Store solves this:
  Single definition of each feature
  Same code runs in batch (training) and streaming (serving)
  Features written once, used by all models

Feature Store Architecture:

  Raw Data (Kafka events, DB tables)
         ■
         ▼
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■               Feature Computation Layer                  ■
  ■  Batch (Spark):    historical features, updated hourly    ■
  ■  Streaming (Flink): real-time features, updated seconds  ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
         ■                           ■
         ▼                           ▼
  ■■■■■■■■■■■■■■■■          ■■■■■■■■■■■■■■■■
  ■  Offline    ■            ■  Online     ■
  ■  Store      ■            ■  Store      ■
  ■  (S3/Hive)  ■            ■  (Redis)    ■
  ■  Training   ■            ■  Serving    ■
  ■  data       ■            ■  (<10ms)    ■
  ■■■■■■■■■■■■■■■■          ■■■■■■■■■■■■■■■■
         ■                           ■
         ▼                           ▼
  Model Training              Model Serving API
  (Spark + TF/PyTorch)        (point lookup by entity_id)

Feature definitions (Feast format):
  feature_view:
    name: user_features
    entities: [user_id]
    features:
      - name: avg_order_value_30d    # average order value, last 30 days
        dtype: FLOAT
      - name: total_orders_7d        # orders placed in last 7 days
        dtype: INT64
      - name: last_category_viewed   # last product category user browsed
        dtype: STRING
    online: True   # materialize to Redis for serving
    ttl: 48h       # Redis key expires after 48 hours
```

Low-Latency Model Serving

```
Inference latency budget (typical recommendation system):
  Total: 100ms for the API response
    Feature retrieval from Redis:   5ms   (batch get for 50 features)
    Model inference:                10ms  (ONNX Runtime on CPU, or 2ms on GPU)
    Post-processing + ranking:      5ms
    DB call for results:            10ms
    Available for other work:       70ms

Optimization techniques:
  1. Model quantization: FP32 weights → INT8 (4× smaller, 3× faster, ~1% accuracy loss)
  2. ONNX Runtime: cross-framework inference (train in PyTorch, serve with ONNX)
  3. Batching: if 100 req/sec arrive, batch 10 together → 1 GPU call
     Saves GPU memory bandwidth — inference cost doesn't scale linearly with batch size
  4. Model caching: cache model predictions for frequent identical inputs
     "user_id=123 seeing product_id=456" → cached for 1 minute
  5. Two-stage retrieval:
     Stage 1 (recall): lightweight model selects 1000 candidates from 1M items (ANN search)
     Stage 2 (ranking): heavy model ranks 1000 candidates precisely
     Much faster than running heavy model on all 1M items

Approximate Nearest Neighbor (ANN) for recommendations:
  Problem: find 20 most similar items to a query vector (embedding) among 10M items
  Exact search: O(10M × embedding_dim) — too slow

  FAISS (Facebook AI Similarity Search):
    Build an index offline: cluster vectors into cells (IVF index)
    Query: find which cell the query belongs to, search only that cell
    Sublinear time: O(sqrt(N)) instead of O(N)
    Trade-off: ~5% of nearest neighbors are missed (approximate)
    Acceptable for recommendations — being 95% accurate is fine

  HNSW (Hierarchical Navigable Small World):
    Graph-based ANN, navigable like a skip list
    Higher quality than IVF but more memory
    Used by: Weaviate, Milvus, Pinecone
```

Chapter 13: A/B Testing & Experimentation Infrastructure

Q13 ■ ■ How do you design an A/B testing platform at Apple's scale?

```
Why A/B testing matters at Apple:
  1% improvement in App Store conversion rate = hundreds of millions in annual revenue
  Every major feature ships with an experiment to measure real user impact
  Data-driven decisions replace opinion-based arguments

Core requirements:
  Consistent assignment: user always sees same variant (not flipping between A/B)
  Mutual exclusion: user in experiment X not in conflicting experiment Y
  Statistical significance: enough traffic per variant to detect real effects
  Minimal latency: assignment decision must add < 1ms to request handling
  Guardrail metrics: automatically detect regressions even if target metric improves

Assignment (the hard part):
  Hash-based (stateless, no DB lookup):
    variant = hash(user_id + experiment_id) % 100
    0-49 → Control (A), 50-99 → Treatment (B)
    Deterministic: same user always gets same variant
    Consistent across services: any service computing same hash gets same result
    No Redis lookup needed → zero latency added

  Overrides: for QA, executives, specific users:
    Check override table FIRST (Redis lookup)
    If override exists → use it (bypasses hash)
    Else → use hash

Experiment configuration (stored in Feature Flag service):
  {
    "experiment_id": "checkout_button_color_v2",
    "status": "running",
    "start_time": "2026-05-01T00:00:00Z",
    "end_time": "2026-05-14T00:00:00Z",
    "traffic_allocation": 10,   // 10% of users in this experiment (rest unaffected)
    "variants": [
      { "id": "control",   "weight": 50, "config": { "button_color": "blue" } },
      { "id": "treatment", "weight": 50, "config": { "button_color": "green" } }
    ],
    "target_metrics": ["checkout_conversion_rate", "add_to_cart_rate"],
    "guardrail_metrics": ["error_rate", "page_load_p99"]
  }

Experiment traffic flow:
  Assignment → logging → analysis

  1. At request time:
     variant = assign(user_id, "checkout_button_color_v2")  // hash-based, ~0.1ms
     log_exposure({ user_id, experiment_id, variant, timestamp })  // async, fire-and-forget

  2. When user converts (adds to cart, checks out):
     log_event({ user_id, event: "checkout_complete", timestamp, revenue: 99.99 })
```

```
  3. Analysis (offline, Spark or BigQuery):
     JOIN exposure logs with event logs on user_id
     Compute: conversion_rate per variant
     Apply statistical test (t-test or Bayesian analysis)
     Check: is difference statistically significant? Is guardrail OK?

Statistical rigor:
  Sample Size Calculation (before starting):
    Effect size: "we want to detect a 2% relative lift in conversion"
    Power: 80% (accept 20% chance of missing a real effect)
    Significance: p < 0.05 (5% false positive rate)
    → Calculator gives: need 50,000 users per variant before analyzing

  Peeking problem: if you check results every day and stop when p < 0.05,
    you're doing multiple comparisons → inflated false positive rate
    Fix: Sequential testing (always-valid p-values) or pre-commit to analysis date

  CUPED (Controlled experiment Using Pre-Experiment Data):
    Reduce variance by controlling for pre-experiment user behavior
    User A has 10% conversion pre-experiment → expected to be higher anyway
    CUPED adjusts: removes pre-existing differences → detects smaller effects
    Commonly reduces required sample size by 40%
```

Part D — Principal-Level Design Problems

Chapter 14: Design Google Spanner

Q14 ■ ■ Design a globally distributed, strongly consistent SQL database

Requirements

```
Functional:
  - Full SQL: SELECT, INSERT, UPDATE, DELETE, JOINs, transactions
  - ACID transactions spanning multiple rows and tables
  - Strong consistency (linearizability) — globally
  - Active in 5 regions: US-West, US-East, EU, India, APAC

Non-functional:
  - 1M writes/sec, 10M reads/sec globally
  - Write latency: < 50ms (cross-region commits)
  - Read latency: < 10ms (local reads), < 50ms (global strong reads)
  - 99.999% availability
  - Survive loss of any single region
```

Architecture

```
Data Sharding:
  Table range-partitioned by primary key (like Bigtable)
  Each range ("tablet") managed by one Paxos group (5 replicas across regions)

  users table (1B rows):
  Tablet 1: user_id 0 – 100M        → Paxos Group 1 (leader: US-West)
  Tablet 2: user_id 100M – 200M     → Paxos Group 2 (leader: US-East)
  Tablet 3: user_id 200M – 300M     → Paxos Group 3 (leader: EU)
  ...

  Each Paxos Group replica: one in each of 5 regions
  Writes: leader handles write, replicates to majority (3 of 5) before committing
  Reads from leader: always fresh (linearizable), ~5ms (local)
  Reads from follower: use safe timestamp to avoid stale reads, ~5ms (local)

Query execution:
  Client sends SQL to any node (coordinator)
  Coordinator parses SQL, creates distributed execution plan
  If query spans multiple tablets: coordinator fans out to multiple tablet leaders
  Results merged at coordinator, returned to client
  Cost: one cross-shard query = multiple network hops (add ~5-10ms per hop)

Schema: interleaved tables (locality optimization)
  CREATE TABLE Users (UserId INT64, Name STRING) PRIMARY KEY (UserId);
  CREATE TABLE Orders (UserId INT64, OrderId INT64, Amount FLOAT64)
    PRIMARY KEY (UserId, OrderId),
    INTERLEAVE IN PARENT Users ON DELETE CASCADE;

  User 123's row and all their Order rows stored in same tablet
  → JOIN on UserId = single tablet read (no cross-shard scatter)
  Critical design pattern: co-locate parent and child tables for common access patterns

Cross-region transaction (worst case):
  BEGIN TRANSACTION
    UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;  // Tablet in US
    UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;  // Tablet in EU
  COMMIT

  1. Coordinator (US-West leader) locks both rows, prepares both Paxos groups
  2. 2PC Prepare:   US tablet ACK (5ms),  EU tablet ACK (100ms RTT + 5ms)
  3. TrueTime wait: ~7ms (2ε uncertainty)
  4. 2PC Commit:    send commit to both, acknowledge to client
  5. Total: ~120ms (dominated by US→EU RTT)

  Optimization: if both accounts were in same region → ~20ms
  Design principle: keep frequently-transacting entities in same region
```

Chapter 15: Design Apple TV+ Live Streaming

Q15 ■ ■ Design a live video streaming platform for millions of concurrent viewers

Requirements

```
Functional:
  - Ingest live video from broadcast cameras (Apple events, sports)
  - Transcode to multiple bitrates (adaptive streaming)
  - Deliver to iOS, Mac, Apple TV, web clients globally
  - < 10 second end-to-end latency (broadcast quality)
  - DVR: pause, rewind up to 4 hours of live stream
  - Concurrent viewers: 50M for major Apple events

Non-functional:
  - 99.99% availability
  - Adaptive bitrate: auto-adjusts based on client bandwidth
  - Graceful degradation: if client bandwidth drops, reduce quality not buffer
```

Video Pipeline

```
Camera → Encoder → RTMP Ingest → Transcoder → Packager → CDN → Client

Step 1: Video Ingest
  Camera sends raw H.264 stream over RTMP to ingest servers
  Ingest servers (multiple for redundancy):
    Primary: receives camera feed
    Backup: hot standby (takes over in < 2 seconds on primary failure)
  Load balancer (Anycast DNS) routes camera to nearest ingest server

Step 2: Transcoding (most compute-intensive step)
  Input: single 1080p 60fps stream (raw or H.264)
  Output: multiple renditions (resolutions × codecs):
    2160p (4K): H.265/HEVC 15Mbps  → Apple TV 4K
    1080p:      H.264 4Mbps        → iPhone on WiFi
    720p:       H.264 2Mbps        → iPad on cellular
    480p:       H.264 1Mbps        → older devices / poor connection
    360p:       H.264 0.5Mbps      → very poor connection fallback

  Transcoding farm: GPU-accelerated (NVIDIA A100 or Apple Silicon)
  Segment duration: 2 seconds (short = lower latency, more files)
  Each segment: independent H.264/HEVC encoded file

Step 3: Packaging (HLS — HTTP Live Streaming)
  Apple's own protocol (now an open standard)
  Master playlist (m3u8): lists all available renditions
    #EXTM3U
    #EXT-X-STREAM-INF:BANDWIDTH=4000000,RESOLUTION=1920x1080
    https://cdn.apple.com/live/1080p.m3u8
    #EXT-X-STREAM-INF:BANDWIDTH=2000000,RESOLUTION=1280x720
    https://cdn.apple.com/live/720p.m3u8

  Per-rendition playlist (live.m3u8, rolling window of last 30 segments):
    #EXTM3U
    #EXT-X-MEDIA-SEQUENCE:100       // segment numbers (for DVR offset)
    segment100.ts                   // 2-second segment
    segment101.ts
    segment102.ts                   // client plays this → ~6s of latency (3 segments)
    #EXT-X-TARGETDURATION:2

  Client: downloads master playlist → picks best rendition → downloads segments
  Adaptive: if download time of segment > 2s → switch down to lower bitrate
            if download time of segment < 0.5s → switch up to higher bitrate

Step 4: CDN Distribution (the scale solution)
  50M viewers × 4Mbps (1080p) = 200 Tbps peak — impossible from origin alone
  CDN (Cloudflare / Akamai / Apple's own CDN): thousands of edge servers worldwide

  Segment caching strategy:
    Popular segments (last 30) → cached at ALL edge servers (hot)
    Older DVR segments → cached at regional PoPs only
```

```
    Archive (> 4 hours) → S3 only (fetched from origin on cache miss)

  Cache TTL: 2 seconds (segment duration) — always fresh
  Cache warming: CDN starts fetching new segments proactively (push model)
    Transcoder → webhook → CDN → prefetch new segment before viewers request it
    Reduces origin hit rate from 100% to < 0.1% for popular live events

DVR (Pause and Rewind):
  All segments written to S3 with retention: s3://live/event/segment{N}.ts
  Client playlist includes historical segment numbers
  Client requests segment 50 (30 min ago) → CDN serves from S3 cache
  DVR window: 4 hours = 7200 segments (2s each) × 10MB avg = 72GB per rendition
```

Chapter 16: Design a Time-Series Database

Q16 ■ ■ Design a time-series database like Apple Health or InfluxDB

Requirements

```
Functional:
  - Ingest: write (metric_name, tags, value, timestamp) at high rate
  - Query: SELECT avg(heart_rate) WHERE user_id=123 AND time > NOW()-7d GROUP BY 1h
  - Retention policies: raw data 30 days, 1-min rollups 1 year, hourly rollups forever
  - Tag-based filtering: WHERE device_type='Apple Watch' AND user_id='...'

Scale:
  - 500M Apple Watch / iPhone users × 10 metrics × 1 write/sec = 5B writes/sec
  - Query: single user's data across 1 year in < 200ms
  - Storage: 500M users × 10 metrics × 86400 samples/day × 30 days × 8B = ~10 PB raw
```

Core Data Model Decisions

```
Design principle: time-series data has special access patterns
  Writes are always "now" (append-only, sequential by time within a series)
  Reads are always time-range queries (rarely point lookups)
  Old data queried rarely, recent data queried constantly
  High cardinality: user_id has 500M unique values

Data model:
  Measurement: "heart_rate"
  Tags (indexed, low cardinality per dimension): { device_type: "Apple Watch 9" }
  Fields (not indexed, actual values): { bpm: 72 }
  Timestamp: unix nanoseconds

  Series = unique combination of (measurement + all tag values)
  "heart_rate,user_id=abc123,device_type=watch9" → one time series

  Time Series ID → time-ordered list of (timestamp, value) pairs

Storage layout (InfluxDB / TimescaleDB approach):
  Chunk data by time range:
    users_heart_rate_2026_05_07  → all heart rate data for May 7
    users_heart_rate_2026_05_06  → all heart rate data for May 6

  Within a chunk, data is sorted by (series_id, timestamp):
    series_abc123: [(t1, 72), (t2, 73), (t3, 71), ...]  ← contiguous on disk
    series_xyz456: [(t1, 65), (t2, 68), ...]

  Benefits:
    Time-range queries read contiguous data (sequential I/O, fast)
    Old chunks dropped atomically (retention policy = drop entire chunk file)
    New data in latest chunk (in memory, flush to disk periodically)
    Compression: delta encoding (store Δtime, Δvalue instead of absolute values)
      72, 73, 71, 74, 72... → Δ: +1, -2, +3, -2 → smaller numbers → better compression
      Gorilla compression (Facebook): 2 hours of data at 1 sample/sec → ~1.37 bytes/sample
      (vs 16 bytes raw for timestamp + float)
```

Architecture

```
Ingest Path (write-heavy — 5B writes/sec):
  ┌──────────────────┐
  │  Health App  │ → gRPC batch write (100 points per request = 50M req/sec)
  └──────────────────┘
         │
  ┌──────▼──────────────────────────────────────┐
  │            Ingest Tier (stateless)       │
  │  Parse, validate, tag normalization      │
  │  Route to correct storage shard          │
  └──────────────────────────────────────────┘
         │  Kafka partitioned by series_id (keeps one series on one consumer)
  ┌──────▼──────────────────────────────────────┐
  │          Storage Tier                    │
  │  WAL → MemTable (sorted by series+time) │
  │  Flush → compressed chunk files (S3)    │
  │  LSM-like compaction (merge small chunks)│
  └──────────────────────────────────────────┘

Query Path:
  ┌──────────────────────────────────────────────────┐
  │  Query: avg(heart_rate) for user=abc123, 7d  │
  └──────────────────────────────────────────────────┘
         │
  ┌──────▼──────────────────────────────────────────┐
  │  Query Planner                                 │
  │  1. Resolve series_id from tags                │
  │  2. Find chunks containing last 7 days         │
  │  3. Prune: chunk min_time > query_end → skip │
  │  4. Fan out to storage nodes holding chunks  │
  └──────────────────────────────────────────────────┘
         │
  ┌──────▼──────────────────────────────────────────┐
  │  Storage Nodes                                 │
  │  Decompress chunks → compute partial avg     │
  │  Return (sum, count) to query planner          │
  └──────────────────────────────────────────────────┘
         │
  ┌──────▼──────────────────────────────────────────┐
  │  Merge partial results → final avg           │
  │  Apply retention: use pre-rolled-up data     │
  │  for data > 30 days old (not raw samples)    │
  └──────────────────────────────────────────────────┘

Rollup pipeline (reduces storage and speeds old queries):
  Kafka Streams job running continuously:
    For every series, every minute:
      Compute: min, max, avg, sum, count of last minute's raw samples
      Write to "1m_rollup" table
    For every series, every hour:
      Aggregate 1m rollups → write to "1h_rollup" table
```

```
  Query for data > 30 days: read from 1h_rollup (1 point/hour vs 3600 points/hour)
  Storage reduction: 30 days raw + 1 year 1m rollup + forever 1h rollup
    = 30 × 3600 + 365 × 60 + ∞ × 1 samples per series per year
    >> 95% storage reduction vs raw data forever
```

Chapter 17: Design a Feature Flag & Experimentation Platform

Q17 ■ ■ Design a feature flag platform used by 50 engineering teams

Requirements

```
Functional:
  - Boolean flags: "is new checkout enabled for this user?"
  - Percentage rollouts: "enable for 5% of users"
  - Targeted rollouts: "enable for users in US only" / "for beta testers"
  - Kill switches: "disable feature instantly if incident detected"
  - A/B test assignment: consistent, bucketed, mutually exclusive
  - SDK for: iOS, Android, Java, Python, Go (< 1ms flag evaluation)

Scale:
  - 10,000 flags across 50 teams
  - 1B flag evaluations/second (every API request evaluates 10-50 flags)
  - Flag changes propagate to all services in < 30 seconds
  - 99.999% availability (flag evaluation must work even if flag service is down)
```

Core Architecture

```
Control Plane (low traffic, high consistency):
  Flag management UI → Flag Config Service → PostgreSQL → Change event → Kafka

Serving Plane (high traffic, low latency):
  SDK reads from local cache, NO remote call per evaluation

  Key insight: Evaluating a flag must be LOCAL and instant.
    Don't call a remote service per flag evaluation — at 1B/sec that's
    impossible and adds latency to every request.

  Solution: push config to every service, evaluate locally.

Config distribution:
  Flag Config Service → publishes config snapshot to Kafka on every change
  SDK (in each service): maintains in-memory copy of all flag configs
    Subscribes to Kafka topic → updates local copy on change
    Evaluation: pure in-memory computation (hash user_id → is it in rollout?)
    Latency: 0.1ms (no network call)
    If Kafka is down: serve from last-known-good in-memory state

Flag evaluation algorithm:
  def evaluate(flag_id, user_id, user_context):
    flag = local_cache[flag_id]   // in-memory lookup

    if not flag.enabled:          // global kill switch
        return flag.default_value

    for rule in flag.rules:       // ordered targeting rules
        if rule.matches(user_context):  // e.g., country == "US"
            return rule.value

    // Percentage rollout (consistent hash)
    bucket = hash(user_id + flag_id) % 10000   // 0-9999
    if bucket < flag.rollout_percentage * 100:  // e.g., 500 for 5%
        return flag.treatment_value

    return flag.default_value     // not in rollout

Kill switch (incident response):
  Engineer sets flag.enabled = false in UI
  Config Service immediately publishes updated config to Kafka
  All services receive update within 30 seconds (Kafka consumer lag)
  SDKs switch to default value instantly on receiving update
  No deploy required — pure config change
```

Multi-Team Flag Governance

```
Namespace isolation:
  "checkout_team.new_promo_engine" → owned by checkout team
  "search_team.ml_ranking_v3"     → owned by search team
  RBAC: each team can only modify their own flags

Flag lifecycle management:
  NEW → ACTIVE → ROLLED_OUT → ARCHIVED
  Automated: if flag has been at 100% rollout for > 30 days → alert team to clean up
  Technical debt: 10,000 stale flags → CPU wasted evaluating dead code
  Quarterly: automated PR to remove flags in ARCHIVED state + their code

Emergency override protocol:
  VP+ can override any flag for ANY user (for live demos, incident response)
  Stored in separate emergency_overrides table (checked first, before normal evaluation)
  Audited: who overrode what, when, for whom
```

Chapter 18: Design Zero-Downtime Database Migration

Q18 ■ ■ How do you migrate 10 billion rows across databases with zero downtime?

Scenario

```
Current state: 10B orders in MySQL (legacy, hitting scale limits)
Target state:  10B orders in CockroachDB (distributed SQL, horizontally scalable)
Constraint:    zero downtime, zero data loss, instant rollback capability
```

Migration Strategy: Dual-Write + Live Migration

```
Phase 1: Read from MySQL, Dual-Write to both
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■  App Code v2                                      ■
  ■  Writes: MySQL (primary) + CockroachDB (secondary) ■
  ■  Reads:  MySQL only                               ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  Duration: 1-2 weeks (stability period)
  Goal: CockroachDB stays in sync with all new writes

Phase 2: Historical data backfill (background job)
  SELECT * FROM mysql.orders WHERE id BETWEEN 0 AND 1000000 ORDER BY id;
  → INSERT into CockroachDB (batches of 1000, with rate limiting)
  → 10B rows ÷ 10,000 rows/sec = ~11.5 days (run during low-traffic hours if needed)
  → Verify checksum of each batch: both DBs agree on count + hash

Phase 3: Verify consistency (shadow reads)
  For 1% of read requests:
    Read from MySQL (serve to user)
    ALSO read from CockroachDB (discard result, but compare)
    If mismatch: log discrepancy (never surface to user)
    Fix discrepancies found in comparison

  Goal: shadow reads show < 0.001% mismatch rate

Phase 4: Switch reads to CockroachDB (still dual-write)
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■  App Code v3                                      ■
  ■  Writes: MySQL (primary) + CockroachDB (secondary) ■
  ■  Reads:  CockroachDB (with fallback to MySQL)     ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  Canary: 1% → 5% → 25% → 100% of read traffic over 1 week
  Rollback: flip read config back to MySQL (no re-migration needed)
  Monitor: error rates, latency, query plan regressions

Phase 5: Switch writes to CockroachDB (still dual-write to MySQL)
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■  App Code v4                                      ■
  ■  Writes: CockroachDB (primary) + MySQL (secondary) ■
  ■  Reads:  CockroachDB                              ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  MySQL is now the secondary — still getting writes for rollback safety
  Monitor for 2 weeks: any issue → flip primary back to MySQL

Phase 6: Decommission MySQL
  Stop dual-write to MySQL
  Verify: CockroachDB has all data (final consistency check)
  Keep MySQL in read-only mode for 30 days (emergency rollback window)
  Decommission MySQL cluster

Timeline: 6-8 weeks total, zero downtime at every phase
Rollback: available at every phase (instant config flip)
```

Chapter 19: Design a Distributed Tracing System

Q19 ■ ■ Design a distributed tracing system like Jaeger/Zipkin at Apple scale

Requirements

```
Functional:
  - Collect traces from 1000+ microservices
  - Trace: sequence of spans (one per service call) with timing + metadata
  - Query: search by trace_id, service, latency > Xms, error = true
  - Flamegraph visualization: which service is the bottleneck?
  - Sampling: don't trace 100% of requests (too expensive)

Scale:
  - 500K requests/sec across services
  - Each request touches 10 services avg → 5M spans/sec
  - Store last 7 days of traces
  - Query latency: < 2s for trace lookup by id
```

Architecture

```
Instrumentation (in each service — via OpenTelemetry SDK):
  Auto-instrumentation: no code change (byte-code agent or sidecar)
  Creates spans for: incoming HTTP, outgoing HTTP, DB queries, cache calls
  Propagates context via HTTP headers: traceparent: 00-traceId-spanId-flags

Trace Collection Pipeline:

  Service A, B, C... → OpenTelemetry Collector (sidecar per node)
                     → Kafka topic "traces" (100 partitions)
                     → Trace Consumer (validates, enriches, stores)
                     → Cassandra (raw spans, TTL 7 days)
                     → Elasticsearch (trace metadata index for search)

Sampling (critical — can't store 100%):
  Head-based sampling: decide AT the root span whether to trace
    Random 1%: sample 1 in 100 requests regardless of outcome
    Problem: misses rare errors (only 0.01% of requests error)

  Tail-based sampling: collect ALL spans temporarily, decide AFTER completion
    If trace has an error → KEEP it (100% of errors sampled)
    If trace is slow (P99+) → KEEP it (100% of slow traces sampled)
    If trace is normal → keep 0.1%
    How: buffer spans in memory for 5s (wait for trace to complete)
         Apply sampling decision → keep or discard
    Cost: more memory (5s × 5M spans/sec × 1KB = 25GB buffer — distributed across collector fleet)

Storage design:
  Cassandra schema:
    TABLE spans (
      trace_id    TEXT,
      span_id     TEXT,
      parent_id   TEXT,
      service     TEXT,
      operation   TEXT,
      start_time  TIMESTAMP,
      duration_ms INT,
      tags        MAP<TEXT,TEXT>,   // { "http.status": "200", "db.table": "users" }
      logs        LIST<TEXT>,
      PRIMARY KEY (trace_id, span_id)
    ) WITH default_time_to_live = 604800;  -- 7 days TTL

    -- Lookup by trace_id: O(1) (partition key)
    -- Fetch all spans: SELECT * FROM spans WHERE trace_id = ?

  Elasticsearch index (for search):
    { trace_id, root_service, root_operation, total_duration_ms, has_error,
      start_time, services_involved: ["order-svc", "payment-svc"] }

    Search: WHERE has_error=true AND services_involved="payment-svc" AND duration_ms > 500
    → Returns matching trace_ids
    → Fetch full spans from Cassandra by trace_id
```

Chapter 20: Design a Multi-Tenant SaaS Platform

Q20 ■ ■ Design a multi-tenant SaaS platform. How do you isolate tenants safely?

Requirements

```
Context: Apple Business Manager — platform used by enterprises to manage
Apple devices. Each enterprise = one tenant. Tenants range from 10-person
startups to Apple Inc itself (500K devices).

Tenants must be:
  - Isolated: Tenant A cannot see Tenant B's data (even by accident)
  - Independent: slow query by Tenant A must not affect Tenant B
  - Configurable: Tenant A has own settings, quotas, feature flags
  - Scalable: some tenants have 100 devices, some have 500K

Non-functional:
  - Data isolation: 100% (compliance requirement)
  - Latency: < 200ms for any tenant query
  - Cost-efficient: not one DB per tenant (too expensive for small tenants)
```

Tenant Isolation Models

```
Model 1: Shared Database, Shared Schema (with tenant_id column)
  Single DB, every table has tenant_id column
  WHERE clause always includes tenant_id (enforced by ORM layer)

  Pros: cheapest (one DB for all), easy to add tenants
  Cons: one runaway query affects all tenants, one bug = data leak
  Use for: small tenants (<1000 devices) who share a "pool"

Model 2: Shared Database, Separate Schema
  Single DB, one schema per tenant
  Apple_Inc schema: devices, users, policies (tables)
  Acme_Corp schema: devices, users, policies (same tables)

  Pros: schema-level isolation (no tenant_id bugs), shared DB infra
  Cons: connection pool per schema, DB becomes a bottleneck at N=1000 tenants
  Use for: mid-size tenants who need isolation but not dedicated DB

Model 3: Dedicated Database per Tenant (silo model)
  Large tenant gets their own DB cluster
  Full isolation: separate connections, separate disk, separate compute

  Pros: blast radius limited to one tenant, dedicated performance, compliance-friendly
  Cons: expensive, hard to operate at scale (1000 DB clusters)
  Use for: large enterprises, regulated industries (government, healthcare)

Hybrid approach (most production systems):
  Tier 1 (< 100 devices):  shared DB + shared schema (pool A)
  Tier 2 (100-10K devices): shared DB + separate schema (pool B)
  Tier 3 (> 10K devices):  dedicated DB cluster
  Tenant routing: tenant_id → tier → connection string (stored in control plane DB)

  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■  Tenant Router (based on tenant_id + tier)             ■
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ▼                     ▼                     ▼
  ■■■■■■■■■■■■■        ■■■■■■■■■■■■■        ■■■■■■■■■■■■■
  ■ Pool DB  ■         ■ Pool DB  ■         ■ Tenant   ■
  ■ (Tier 1) ■         ■ (Tier 2) ■         ■ DB (T3)  ■
  ■ Shared   ■         ■ Schema   ■         ■ Dedicated■
  ■ schema   ■         ■ isolation■         ■ cluster  ■
  ■■■■■■■■■■■■■        ■■■■■■■■■■■■■        ■■■■■■■■■■■■■

Noisy neighbor protection (Tier 1 pools):
  Per-tenant query timeout: 10 seconds max (even if pool allows longer)
  Per-tenant connection limit: max 5 connections from tenant's app server
  Resource quotas: max 100 API req/sec per tenant (rate limiting at API gateway)
  Query monitoring: alert if tenant's query P99 > 500ms (might indicate bad query)
```

Tenant Onboarding Automation

```
New tenant signup → fully automated provisioning:
  1. Create tenant record in control plane DB
  2. Determine tier (based on contract/device count estimate)
  3. If Tier 1/2: create schema in appropriate pool DB
                  run schema migrations for this tenant
  4. If Tier 3:   provision new DB cluster (AWS RDS or Aurora)
                  run schema migrations
                  create connection credentials
  5. Store tenant config (tier, DB endpoint, schema, API quotas) in control plane
  6. Propagate to API gateway (tenant routing table)
  7. Send welcome email → tenant is live

Migration between tiers (when tenant grows):
  Tier 1 → Tier 2: copy schema out of shared schema, create dedicated schema
  Tier 2 → Tier 3: dump tenant's schema, restore to dedicated DB, update routing
  No downtime: dual-write during migration, verify, switch, cleanup
```

Principal Engineer Quick Reference

When interviewers ask "what does a Principal Engineer bring differently?"

```
Staff Engineer answer:     "I designed a system that handles 1M req/sec"
Senior Staff answer:       "I designed it with CAP/PACELC trade-offs, Saga pattern,
                            and CRDT for conflict resolution"
Principal Engineer answer: "I designed the PLATFORM that 40 teams use to build their
                            own systems safely. I defined the cell-based isolation strategy
                            that limits blast radius. I wrote the schema migration runbook
                            that all teams follow. I drove the decision to adopt Spanner
                            for our payment ledger — here's the build vs buy analysis
                            and the 5-year cost projection. I aligned 5 engineering orgs
                            around a consistent observability stack."

Key indicators of Principal-level thinking:
  ✓ System evolution: "in 2 years when X happens, the design needs to..."
  ✓ Org impact: "this decision affects how 50 teams build software"
  ✓ Quantified trade-offs: "$2M/month more but saves 40% engineering time — worth it"
  ✓ Failure mode exhaustion: runs through every component's failure scenario
  ✓ Build vs buy: knows when to use open source vs build vs buy managed service
  ✓ Adjacent concerns: security, compliance, cost, operability — not just correctness
```

Deep-Dive Topics Comparison Across Levels

Topic	*Staff (Vol I)*	*Senior Staff (Vol II)*	*Principal (Vol III)*
Storage	"Use PostgreSQL for ACID"	"LSM vs B-Tree trade-off"	MVCC internals, WAL structure, VACUUM

Topic	*Staff (Vol I)*	*Senior Staff (Vol II)*	*Principal (Vol III)*
Caching	LRU/LFU policies	L1/L2 cache, Redis cluster	Bloom filters, Count-Min Sketch, HLL
Distributed DB	Sharding, replication	CAP, Raft, CockroachDB	TrueTime, Spanner, cross-region ACID
Migrations	Schema changes	Schema registry, Avro	Expand-contract, live migration, dual-write
Reliability	Circuit breaker	Chaos theory	Chaos engineering, game days, cell architecture
ML	N/A	Feature store concept	Feature store internals, CUPED, ANN (FAISS)
Experimentation	A/B testing concept	Statistical significance	CUPED, sequential testing, platform governance
Streaming	Kafka basics	Lambda vs Kappa	Tail-based sampling, backpressure, watermarks
Security	JWT basics	mTLS concept	Zero-trust, SPIFFE, service mesh, HSM
Multi-tenancy	N/A	N/A	Silo vs pool model, noisy neighbor, tier routing

---

Prepared for Apple Inc Principal Engineer Interview | System Design Volume III

>

*Principal Engineer themes Apple probes: - **Internals mastery**: LSM vs B-Tree, MVCC, TrueTime — not just "use PostgreSQL" - **Platform thinking**: design for other teams to build on top of, not just one system - **Org-scale decisions**: trade-offs that affect 50 teams, 5-year cost projections - **Operability**: how does this run at 3am? who gets paged? what's the runbook? - **Evolution**: what does this look like in 2 years? 5 years? what must stay stable? - **Failure exhaustion**: every component's failure mode, blast radius, and recovery path - **Build vs buy**: deep knowledge of managed services, open source, and when to build*